Tutorial Works Session 1

Dr. Rochdi Boudjehem

2025-10-12

Part 1: Motivation and Fundamental Concepts

Exercise 1.1: The Need for Parallelism (Conceptual)

1. Explain two main reasons that necessitated the shift from single-core performance improvements to parallel computing approaches.

2. What do you know about **Moore's Law** and **Dennard Scaling**? How do they relate to the evolution of computing hardware? How do they apply in our era of multi-core and many-core processors?

3. In Operating Systems, you used to learn about processes and threads. When checking your CPU specifications, you may have seen terms like "cores" and "threads". What is the difference between software threads and hardware threads (cores)?

4. you want to buy a new computer for high-performance computing tasks such as Machine Learning and Deep Learning:

   - Would you prioritize a CPU with a higher clock speed or one with more cores? Justify your choice based on the concepts discussed in the sources.
   - Would you prioritize a powerful CPU with more cores with a weak GPU, or a lightweight CPU with a powerful GPU? Justify your choice.
   - Would you prioritize an L2 over L3 cache? Justify your choice.
   - Would you consider a system with specialized accelerators (e.g., TPUs) for your tasks? Why or why not?

Exercise 1.2: Execution Models

1. Define the difference between **concurrent execution** and **parallel execution**.

2. Identify whether the following task components are typically sequential or parallel, based on the definitions in the sources:

a. Reading input data. .
b. Splitting input data into chunks.
c. Processor executing the main computation on its chunk.
d. Combining results and writing output data.

Part 2: Performance Metrics and Laws

Exercise 1.3: Basic Performance Calculation

1. A sequential program takes $T_1 = 120$ seconds to execute. When parallelized and run on $T = 8$ workers, the execution time is reduced to $T_T = 30$ seconds.

a. Calculate the Speedup, $S(T)$, defined as $S(T) = \frac{T_1}{T_T}$.
b. Calculate the Efficiency, $E(T)$, defined as $E(T) = \frac{S(T)}{T}$.

2. What unit is frequently used to express the peak performance of a system in terms of floating-point operations?

Exercise 1.4: Applying Amdahl's Law (Fixed Problem Size)

Amdahl's Law states that the speedup of a program using multiple processors in parallel computing is limited by the serial portion of the program. The formula is given by:

$$S(N) = \frac{1}{S + \frac{P}{N}}$$

where:

- S is the fraction of the program that is serial (cannot be parallelized),
- P is the fraction of the program that can be parallelized ($P = 1 - S$),
- N is the number of processors.

Consider a program where the serial fraction (S) is 15 ($f = 0.15$). The remaining portion (P) can be perfectly parallelized.

1. Calculate the theoretical Speedup, $S(N)$, achievable if the program is run using $N = 4$ processors.
2. Calculate the theoretical Speedup, $S(N)$, if the program is run using $N = 32$ processors.
3. Determine the **maximum theoretical speedup** achievable for this program, assuming an infinite number of processors ($N \to \infty$).
4. What can you conclude about the limitations imposed by Amdahl's Law on parallel computing performance?
5. Briefly explain why Amdahl's Law is often considered a "pessimistic" perspective on parallelization.

Exercise 1.5: Inverting Amdahl's Law

Suppose a parallel system with $N = 8$ processors achieves a measured speedup $S(8) = 5.0$ for a given workload.

1. Estimate the serial fraction (S or f) of this program by inverting Amdahl's formula.
2. If the number of processors were increased to $N = 64$ (using the same calculated serial fraction S), calculate the new theoretical speedup.

Exercise 1.6: Gustafson's Law (Scaled Problem Size)

Gustafson's Law provides an alternative perspective to Amdahl's Law by considering scaled problem sizes. It states that the speedup $S(N)$ with N processors can be expressed as:

$$S(N) = N - S \times (N - 1)$$

where:

- N is the number of processors.

- S is the serial fraction of the program.
- P is the parallel fraction of the program ($P = 1 - S$).

Consider a program with a serial fraction of $S = 0.1$ (10%) and a parallel fraction of $P = 0.9$ (90%).

1. Using Gustafson's Law, calculate the speedup $S(N)$ when using $N = 16$ processors.
2. For the same program, calculate the speedup $S(N)$ when using $N = 64$ processors.
3. Compare the results from Gustafson's Law with those from Amdahl's Law for the same serial fraction and number of processors.
4. Discuss the implications of each law in terms of parallel computing performance.

Exercise 1.7: Applying Gustafson's Law (Scalable Problem Size)

Consider a program that, when run on $N = 16$ processors, spends 15 of its total parallel execution time on sequential tasks ($S = 0.15$).

1. Use Gustafson's Law to calculate the **scaled speedup**, $S_G(N)$.
2. Assuming the same sequential fraction ($f = 0.15$), compare the prediction from Gustafson's Law (scaled problem) with the prediction from Amdahl's Law (fixed problem) for $N = 16$.
3. Explain the key difference in assumption between Amdahl's Law and Gustafson's Law regarding the problem size.

Part 2: Architectural Classification and Systems

Exercise 2.1: Flynn's Taxonomy Definitions

Flynn's taxonomy classifies architectures based on Instruction Streams and Data Streams.

1. Define the four main categories of Flynn's taxonomy using their acronyms.
2. Describe the key operational difference between a **SIMD** architecture and a **MIMD** architecture, and provide a modern example for each.
3. Which category of Flynn's taxonomy is described as "rare and generally of academic interest"?

Exercise 2.2: Classifying HPC Systems

A modern **GPU-accelerated cluster node** uses multiple cores and specialized hardware.

1. Classify this system at the **node/cluster level** using Flynn's taxonomy.
2. Classify the operation *within* the GPU accelerator cores using Flynn's taxonomy.
3. Briefly differentiate between the three main categories of parallel systems discussed in the sources: **shared-memory systems**, **distributed-memory systems**, and **accelerated systems**23.

Exercise 2.3: Programming Models and Architectures (Preview)

Match the following programming models/interfaces to the parallel system category they are primarily intended for:

Programming Models/Interfaces	Parallel System Categories
OpenMP	Shared-Memory Systems

Programming Models/Interfaces	Parallel System Categories
MPI	.....
CUDA/OpenCL	.....
Pthreads	.....

Parallel System Categories:

1. **Shared-Memory Systems**
2. **Distributed-Memory Systems**
3. **Accelerated Systems**